

Coverage Metric - GPU Optimized

Karina Arichavala

2026-05-11

Table of contents

0.1	1. Setup e Imports	1
0.2	2. Cargar Datos	2
0.3	3. Cargar SAE y Extraer Features	3
0.4	4. Filtrar BSPs de Piezas	4
0.5	5. Implementación de Coverage	5
0.5.1	Estrategia de optimización:	5
0.6	6. Calcular Coverage	8
0.7	7. Análisis de Resultados	8
0.8	8. Guardar Resultados	35
0.9	9. Generar PDF con Quarto	35

```
# Configuración para visualización en PDF
import pandas as pd
import numpy as np
import sys

# Para que el texto de pandas no se corte
pd.set_option('display.max_colwidth', None)
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 100) # Ancho fijo para pandas

# Para prints largos de numpy - ancho más conservador
np.set_printoptions(linewidth=100, edgeitems=3)

# Configurar ancho de terminal para prints
import os
os.environ['COLUMNS'] = '100'

print(" Configuración para PDF lista")
```

Configuración para PDF lista

0.1 1. Setup e Imports

```
import numpy as np
import torch
import sys
import os
from pathlib import Path
from tqdm import tqdm
import time

# Configurar paths
project_root = Path('../..').resolve()
sys.path.insert(0, str(project_root))
```

```

from sae.models.sae import SparseAutoencoder
from sae.tools.naming_utils import get_checkpoint_dir, get_model_name, get_extras_id
from sae.tools.experiment_utils import get_experiment_dir, get_metrics_dir, get_report_name

# Verificar GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Device: {device}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"Memoria disponible: {torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")

```

Device: cuda
 GPU: NVIDIA GeForce RTX 5060
 Memoria disponible: 8.55 GB

```

# Metadata para naming de checkpoints (separada de hiperparámetros)
NAMING_META = {
    'variante': 'standard',          # Arquitectura del SAE
    'tecnica': 'l1-decay',          # L1 con penalty annealing
    'capa': 6,                      # Layer del modelo OthelloGPT
    'juegos': 500,                  # Número de partidas
    'base_path': 'C:\\Users\\Esposa\\Documents\\Repos\\sae-othello-gpt', # Path base para checkpoints
    'experiment_base_path': os.path.abspath('.././experiments'), # sae/experiments/
    'extras': {
        'expansion': 32,
        'sparsity_coef': 5e-4,
        'epochs': 200,
        'patience': 15,
        'learning_rate': 1e-3,
        'sparsity_decay': 0.95,
        'batch_size': 512
    }
}
print('\n Metadata del experimento:')
for key, value in NAMING_META.items():
    print(f' {key}: {value}')

```

Metadata del experimento:

```

variante: standard
tecnica: l1-decay
capa: 6
juegos: 500
base_path: C:\Users\Esposa\Documents\Repos\sae-othello-gpt
experiment_base_path: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments
extras: {'expansion': 32, 'sparsity_coef': 0.0005, 'epochs': 200, 'patience': 15, 'learning_rate': 0.001, 'sparsity_c

```

0.2 2. Cargar Datos

```

# Cargar activaciones pre-SAE
activations_path = project_root / "sae" / "activations" / "data" / "layer6_200games.npy"
activations_full = np.load(activations_path)
####
activations = activations_full

print(f"Activaciones del modelo:")
print(f" Shape original: {activations_full.shape}")
print(f" Shape final: {activations.shape}")
print(f" Memoria: {activations.nbytes / (1024**2):.2f} MB")

```

Activaciones del modelo:

Shape original: (11800, 512)

Shape final: (11800, 512)

Memoria: 23.05 MB

```
# Cargar ground truth de BSPs
bsp_gt_path = project_root / "sae" / "metrics" / "02_data" / "bsp_ground_truth_200games.npy"
bsp_names_path = project_root / "sae" / "metrics" / "02_data" / "bsp_ground_truth_200games.names.npy"

bsp_ground_truth_full = np.load(bsp_gt_path)
bsp_names = np.load(bsp_names_path, allow_pickle=True)
###
indices = np.random.choice(len(activations_full), size=1000, replace=False)
activations = activations_full[indices]
##
bsp_ground_truth = bsp_ground_truth_full[indices]
print(f"\nGround truth BSPs:")
print(f" Shape original: {bsp_ground_truth_full.shape}")
print(f" Shape final: {bsp_ground_truth.shape}")
print(f" Total BSPs: {len(bsp_names)}")
```

Ground truth BSPs:

Shape original: (11800, 326)

Shape final: (1000, 326)

Total BSPs: 326

0.3 3. Cargar SAE y Extraer Features

```
# Configuración del SAE
input_dim = 512
hidden_dim = 16384

# Construir ruta del modelo usando naming_utils
checkpoint_dir = get_checkpoint_dir(
    NAMING_META['base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos']
)
extras_id = get_extras_id(checkpoint_dir, NAMING_META['extras']) if NAMING_META.get('extras') else None

model_name = get_model_name(
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    'best',
    extras_id=extras_id
)
model_path = checkpoint_dir / model_name

# Cargar modelo (weights_only=False: checkpoint contiene objetos Path en config)
sae = SparseAutoencoder(input_dim, hidden_dim).to(device)
checkpoint = torch.load(model_path, map_location=device, weights_only=False)
sae.load_state_dict(checkpoint['model_state_dict'])
sae.eval()

print(f"SAE cargado:")
```

```

print(f" Path: {model_path}")
print(f" Input: {input_dim}, Hidden: {hidden_dim}")
print(f" Expansion: {hidden_dim/input_dim}x")
print(f" Epoch: {checkpoint.get('epoch', 'N/A')}, Val MSE: {checkpoint.get('val_mse', float('nan')):.6f}")

```

SAE cargado:

```

Path: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_standard\sae_l1-decay_standard\500games\sae
decay_16_500g_ex5_best.pt
Input: 512, Hidden: 16384
Expansion: 32.0x
Epoch: N/A, Val MSE: nan

```

```

print("Extrayendo features del SAE...")
activations_tensor = torch.from_numpy(activations).float().to(device)

# PRIMERO: calcular sae_features con los 15 tableros
with torch.no_grad():
    sae_features = torch.relu(sae.encoder(activations_tensor))

# SEGUNDO: calcular f_max_global sobre TODO el dataset
activations_full_tensor = torch.from_numpy(activations_full).float().to(device)
with torch.no_grad():
    sae_features_full = torch.relu(sae.encoder(activations_full_tensor))
f_max_global = sae_features_full.max(dim=0)[0]

# Liberar memoria
del activations_full_tensor, sae_features_full
torch.cuda.empty_cache()
print("f_max global calculado sobre todo el dataset")

print(f"\nFeatures SAE:")
print(f" Shape: {sae_features.shape}")
print(f" Device: {sae_features.device}")
print(f" Sparsity: {(sae_features == 0).float().mean():.2%}")
print(f" Activaciones promedio: {(sae_features > 0).sum(dim=1).float().mean():.1f}")

```

Extrayendo features del SAE...

f_max global calculado sobre todo el dataset

Features SAE:

```

Shape: torch.Size([1000, 16384])
Device: cuda:0
Sparsity: 94.01%
Activaciones promedio: 980.6

```

0.4 4. Filtrar BSPs de Piezas

Coverage solo usa las 128 BSPs de piezas (mías/oponente), sin vacías.

```

# =====
# CONFIGURACIÓN DE FILTRADO DE BSPs
# =====
USE_PLAYER   = False # BSPs perspectiva del jugador (1 y 2)
USE_ABSOLUTE = True  # BSPs absolutas negro/blanco (B y W)
USE_STRATEGIC = False # BSPs especiales (BSP_)

# Filtrar según configuración
bsp_piezas_indices = []
bsp_piezas_names = []

```

```

for i, name in enumerate(bsp_names):
    include = False

    if USE_PLAYER and len(name) == 6 and name.startswith('BSP') and (name.endswith('1') or name.endswith('2')):
        include = True
    if USE_ABSOLUTE and (name.endswith('B') or name.endswith('W')):
        include = True
    if USE_STRATEGIC and name.startswith('BSP_'):
        include = True

    if include:
        bsp_pieces_indices.append(i)
        bsp_pieces_names.append(name)

bsp_pieces_indices = np.array(bsp_pieces_indices)
bsp_pieces_gt = bsp_ground_truth[:, bsp_pieces_indices]

# Convertir a tensor en GPU
bsp_pieces_gt_tensor = torch.from_numpy(bsp_pieces_gt).bool().to(device)

print(f"BSPs seleccionadas para Coverage:")
print(f" Player: {USE_PLAYER} | Absolute: {USE_ABSOLUTE} | Strategic: {USE_STRATEGIC}")
print(f" Total: {len(bsp_pieces_indices)}")
print(f" Shape: {bsp_pieces_gt_tensor.shape}")
print(f" Device: {bsp_pieces_gt_tensor.device}")
print(f" Primeras 5: {' ', ' '.join(bsp_pieces_names[:5])}")
print(f" Últimas 5: {' ', ' '.join(bsp_pieces_names[-5:])}")
# Identificador del filtrado (para naming de archivos de salida)
parts = []
if USE_PLAYER: parts.append("player")
if USE_ABSOLUTE: parts.append("absolute")
if USE_STRATEGIC: parts.append("strategic")
filter_suffix = "_".join(parts) if parts else "none"
print(f" Filter suffix: {filter_suffix}")

```

```

BSPs seleccionadas para Coverage:
Player: False | Absolute: True | Strategic: False
Total: 128
Shape: torch.Size([1000, 128])
Device: cuda:0
Primeras 5: BSPA1B, BSPA1W, BSPA2B, BSPA2W, BSPA3B
Últimas 5: BSPH6W, BSPH7B, BSPH7W, BSPH8B, BSPH8W
Filter suffix: absolute

```

0.5 5. Implementación de Coverage

0.5.1 Estrategia de optimización:

1. **Vectorización:** Procesar múltiples features en paralelo usando operaciones matriciales
2. **Batch processing:** Dividir en chunks para no saturar memoria GPU
3. **Pre-cálculo:** Calcular máximos una sola vez
4. **Early stopping:** Terminar cuando F1 = 1.0

```

def fast_f1_score_gpu(y_true, y_pred, eps=1e-8):
    """
    F1 score vectorizado en GPU.

    Args:
        y_true: Tensor (n_positions,) booleano
        y_pred: Tensor (n_positions, n_candidates) booleano
    """

```

```

Returns:
    f1_scores: Tensor (n_candidates,)
"""
# y_true: (n_positions,) -> expand to (n_positions, 1)
y_true_expanded = y_true.unsqueeze(1) # (n_positions, 1)

# Calcular TP, FP, FN
tp = (y_true_expanded & y_pred).sum(dim=0).float() # (n_candidates,)
fp = (~y_true_expanded & y_pred).sum(dim=0).float()
fn = (y_true_expanded & ~y_pred).sum(dim=0).float()

# Precision y Recall
precision = tp / (tp + fp + eps)
recall = tp / (tp + fn + eps)

# F1
f1 = 2 * (precision * recall) / (precision + recall + eps)

return f1

def calculate_coverage_gpu_optimized(
    sae_features, # Tensor (n_positions, n_features) en GPU
    bsp_ground_truth, # Tensor (n_positions, n_bsps) en GPU, bool
    thresholds=[0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9],
    batch_size_features=512, # Procesar 512 features a la vez
    f_max=None, # ← AGREGAR
    verbose=True
):
    """
    Calcula Coverage de forma optimizada para GPU.

    Procesa múltiples features y thresholds en paralelo usando operaciones matriciales.
    """
    n_positions, n_features = sae_features.shape
    n_bsps = bsp_ground_truth.shape[1]
    n_thresholds = len(thresholds)

    if verbose:
        print("="*60)
        print("CALCULATING COVERAGE (GPU OPTIMIZED)")
        print("="*60)
        print(f"Positions: {n_positions:,}")
        print(f"SAE Features: {n_features:,}")
        print(f"BSPs: {n_bsps:,}")
        print(f"Thresholds: {n_thresholds:,}")
        print(f"Batch size (features): {batch_size_features:,}")
        print(f"Total evaluations: {n_bsps * n_features * n_thresholds:,}")
        print("="*60)

    # Pre-calcular máximos de features
    if f_max is None:
        f_max = sae_features.max(dim=0)[0]
        active_features = (f_max > 0).nonzero(as_tuple=True)[0]
        n_active = len(active_features)

    if verbose:
        print(f"\nActive features: {n_active:,}/{n_features:,} ({n_active/n_features*100:.1f}%)")
        print(f"Reduced evaluations: {n_bsps * n_active * n_thresholds:,}")
        print()

    # Almacenar resultados

```

```

best_f1s = torch.zeros(n_bsps, device=sae_features.device)
best_features = torch.full((n_bsps,), -1, dtype=torch.long, device=sae_features.device)
best_thresholds = torch.full((n_bsps,), -1.0, device=sae_features.device)

thresholds_tensor = torch.tensor(thresholds, device=sae_features.device)

# Procesar cada BSP
pbar = tqdm(range(n_bsps), desc="Processing BSPs") if verbose else range(n_bsps)

for bsp_idx in pbar:
    bsp_labels = bsp_ground_truth[:, bsp_idx] # (n_positions,)

    # Skip si la BSP nunca está activa
    if not bsp_labels.any():
        continue

    # Procesar features activas en batches
    for batch_start in range(0, n_active, batch_size_features):
        batch_end = min(batch_start + batch_size_features, n_active)
        batch_features_idx = active_features[batch_start:batch_end]

        # Extraer features del batch
        batch_activations = sae_features[:, batch_features_idx] # (n_positions, batch_size)
        batch_f_max = f_max[batch_features_idx] # (batch_size,)

        # Para cada threshold, binarizar TODAS las features del batch a la vez
        for t in thresholds_tensor:
            # Binarizar: (n_positions, batch_size)
            predictions = batch_activations > (t * batch_f_max.unsqueeze(0))

            # Calcular F1 para todas las features del batch
            f1_scores = fast_f1_score_gpu(bsp_labels, predictions) # (batch_size,)

            # Encontrar la mejor en este batch
            max_f1, max_idx_in_batch = f1_scores.max(dim=0)

            # Actualizar si es mejor que lo visto hasta ahora
            if max_f1 > best_f1s[bsp_idx]:
                best_f1s[bsp_idx] = max_f1
                best_features[bsp_idx] = batch_features_idx[max_idx_in_batch]
                best_thresholds[bsp_idx] = t

        # Early stopping: si ya es casi perfecto, no revisar más batches
        if best_f1s[bsp_idx] >= 0.999:
            break

    # Calcular Coverage (macro-average)
    coverage = best_f1s.mean().item()

    # Convertir a CPU para retornar
    best_f1s_cpu = best_f1s.cpu().numpy()
    best_features_cpu = best_features.cpu().numpy()
    best_thresholds_cpu = best_thresholds.cpu().numpy()

    return coverage, best_f1s_cpu, best_features_cpu, best_thresholds_cpu

print(" Funciones de Coverage GPU definidas")

```

Funciones de Coverage GPU definidas

0.6 6. Calcular Coverage

```
# Medir tiempo de ejecución
start_time = time.time()

coverage, best_f1s, best_features, best_thresholds = calculate_coverage_gpu_optimized(
    sae_features,
    bsp_pieces_gt_tensor,
    batch_size_features=512, # Ajustar según memoria GPU
    f_max=f_max_global,
    verbose=True
)

elapsed_time = time.time() - start_time

print("\n" + "="*60)
print("RESULTADO COVERAGE")
print("="*60)
print(f"Coverage Score: {coverage:.4f}")
print(f"Objetivo (paper): 0.52")
print(f"Diferencia: {coverage - 0.52:+.4f}")
print(f"\nTiempo de ejecución: {elapsed_time:.2f} seg")
print(f"({elapsed_time/60:.2f} min)")
print("="*60)
```

```
=====
CALCULATING COVERAGE (GPU OPTIMIZED)
=====

Positions: 1,000
SAE Features: 16,384
BSPs: 128
Thresholds: 10
Batch size (features): 512
Total evaluations: 20,971,520
=====

Active features: 14,548/16,384 (88.8%)
Reduced evaluations: 18,621,440

Processing BSPs: 100%|      | 128/128 [00:31<00:00, 4.11it/s]

=====
RESULTADO COVERAGE
=====

Coverage Score: 0.4630
Objetivo (paper): 0.52
Diferencia: -0.0570

Tiempo de ejecución: 31.14 seg
                    (0.52 min)
=====
```

0.7 7. Análisis de Resultados


```
# Estadísticas de distribución de F1 scores
print("Distribución de F1 scores:")
print(f" Mínimo: {best_f1s.min():.4f}")
print(f" Máximo: {best_f1s.max():.4f}")
print(f" Media: {best_f1s.mean():.4f}")
print(f" Mediana: {np.median(best_f1s):.4f}")
print(f" Std: {best_f1s.std():.4f}")
print()
print(f"BSPs con F1 > 0.8: {(best_f1s > 0.8).sum()} ({(best_f1s > 0.8).mean()*100:.1f}%)")
print(f"BSPs con F1 > 0.5: {(best_f1s > 0.5).sum()} ({(best_f1s > 0.5).mean()*100:.1f}%)")
print(f"BSPs con F1 < 0.2: {(best_f1s < 0.2).sum()} ({(best_f1s < 0.2).mean()*100:.1f}%)")
```

Distribución de F1 scores:

Mínimo: 0.3164
Máximo: 0.6956
Media: 0.4630
Mediana: 0.4434
Std: 0.0957

BSPs con F1 > 0.8: 0 (0.0%)
BSPs con F1 > 0.5: 38 (29.7%)
BSPs con F1 < 0.2: 0 (0.0%)

```
# Top 10 BSPs mejor detectadas
top_indices = np.argsort(best_f1s)[-10:][::-1]

print("\nTop 10 BSPs mejor detectadas:")
print("="*60)
print(f"{'BSP':<10} {'F1':<8} {'Feature':<10} {'Threshold':<10}")
print("-"*60)
for idx in top_indices:
    bsp_name = bsp_pieces_names[idx]
    f1 = best_f1s[idx]
    feat = best_features[idx]
    thresh = best_thresholds[idx]
    print(f"{bsp_name:<10} {f1:<8.4f} {feat:<10} {thresh:<10.1f}")
```

Top 10 BSPs mejor detectadas:

```
=====
```

BSP	F1	Feature	Threshold
BSPD4B	0.6956	2309	0.0
BSPE4B	0.6913	6068	0.0
BSPE5B	0.6860	10207	0.0
BSPD5B	0.6752	6068	0.0
BSPD5W	0.6728	7238	0.0
BSPD3B	0.6579	6156	0.0
BSPE5W	0.6556	3343	0.0
BSPE4W	0.6498	9762	0.0
BSPD4W	0.6463	10207	0.0
BSPC5B	0.6111	11708	0.0

```
-----
```

```
# Bottom 10 BSPs peor detectadas
bottom_indices = np.argsort(best_f1s)[:10]

print("\nBottom 10 BSPs peor detectadas:")
print("="*60)
print(f"{'BSP':<10} {'F1':<8} {'Feature':<10} {'Threshold':<10}")
print("-"*60)
for idx in bottom_indices:
```

```

bsp_name = bsp_pieces_names[idx]
f1 = best_f1s[idx]
feat = best_features[idx]
thresh = best_thresholds[idx]
print(f"{bsp_name:<10} {f1:<8.4f} {feat:<10} {thresh:<10.1f}")

```

Bottom 10 BSPs peor detectadas:

```

=====
BSP          F1          Feature    Threshold
-----
BSPB8B       0.3164       2944         0.4
BSPG8B       0.3322       9630         0.1
BSPA1B       0.3333       10594        0.4
BSPA2B       0.3355       3529         0.4
BSPF8B       0.3384       9348         0.3
BSPC1W       0.3410       9630         0.2
BSPH6B       0.3423       9630         0.2
BSPH2B       0.3436       14661        0.2
BSPA4B       0.3439       4890         0.4
BSPH7B       0.3480       9630         0.1

```

```

# Distribución de thresholds óptimos
unique_thresholds, counts = np.unique(best_thresholds, return_counts=True)

print("\nDistribución de thresholds óptimos:")
print("=*60)
for t, count in zip(unique_thresholds, counts):
    if t >= 0: # Ignorar -1 (BSPs sin match)
        percentage = count / len(best_thresholds) * 100
        print(f"Threshold {t:.1f}: {count:3d} BSPs ({percentage:.1f}%)")

```

Distribución de thresholds óptimos:

```

=====
Threshold 0.0: 16 BSPs (12.5%)
Threshold 0.1: 42 BSPs (32.8%)
Threshold 0.2: 43 BSPs (33.6%)
Threshold 0.3: 18 BSPs (14.1%)
Threshold 0.4: 8 BSPs (6.2%)
Threshold 0.5: 1 BSPs (0.8%)

```

```

# =====
# CONSULTA POR BSP ESPECÍFICA
# =====

# >>> CAMBIA AQUÍ LA BSP QUE QUIERES ANALIZAR <
BSP_OBJETIVO = "BSPE4B"

# Buscar índice de la BSP
bsp_idx = bsp_pieces_names.index(BSP_OBJETIVO)

# Extraer resultados ya calculados
neurona = best_features[bsp_idx]
threshold = best_thresholds[bsp_idx]
f1 = best_f1s[bsp_idx]

# f_max de la neurona ganadora (sae_features ya está en memoria)
f_max_neurona = sae_features[:, neurona].max().item()

# Activaciones de esa neurona en todos los tableros

```

```

activaciones_neurona = sae_features[:, neurona].cpu().numpy()

print(f"BSP analizada:      {BSP_OBJETIVO}")
print(f"Neurona ganadora:   {neurona}")
print(f"Threshold óptimo:     {threshold:.1f}")
print(f"F_max neurona:        {f_max_neurona:.4f}")
print(f"Umbral efectivo:       {threshold:.1f} x {f_max_neurona:.4f} = {threshold * f_max_neurona:.4f}")
print(f"F1 score:             {f1:.4f}")
print(f"\nActivaciones de neurona {neurona} ({len(activaciones_neurona)} tableros):")
for i, act in enumerate(activaciones_neurona):
    print(f"  Tablero {i+1:5d}: {act:.6f}")

```

```

BSP analizada:      BSPE4B
Neurona ganadora:   6068
Threshold óptimo:    0.0
F_max neurona:       10.3025
Umbral efectivo:     0.0 x 10.3025 = 0.0000
F1 score:            0.6913

```

Activaciones de neurona 6068 (1000 tableros):

```

  Tablero    1: 0.416347
  Tablero    2: 0.472797
  Tablero    3: 0.299994
  Tablero    4: 0.840389
  Tablero    5: 0.867366
  Tablero    6: 0.786457
  Tablero    7: 0.417086
  Tablero    8: 0.576797
  Tablero    9: 1.198605
  Tablero   10: 0.562271
  Tablero   11: 0.515096
  Tablero   12: 0.468794
  Tablero   13: 0.562849
  Tablero   14: 0.526186
  Tablero   15: 1.190435
  Tablero   16: 0.650913
  Tablero   17: 0.765988
  Tablero   18: 0.829102
  Tablero   19: 0.871225
  Tablero   20: 0.848128
  Tablero   21: 0.481537
  Tablero   22: 0.727437
  Tablero   23: 1.058713
  Tablero   24: 0.964400
  Tablero   25: 0.815916
  Tablero   26: 0.000000
  Tablero   27: 0.974014
  Tablero   28: 0.945982
  Tablero   29: 0.685446
  Tablero   30: 0.887038
  Tablero   31: 0.642986
  Tablero   32: 0.700627
  Tablero   33: 0.716990
  Tablero   34: 0.677816
  Tablero   35: 0.402941
  Tablero   36: 0.699234
  Tablero   37: 2.122669
  Tablero   38: 0.302952
  Tablero   39: 0.875976
  Tablero   40: 1.129833
  Tablero   41: 0.169220
  Tablero   42: 0.608311

```

Tablero	43:	0.399515
Tablero	44:	0.446879
Tablero	45:	0.656681
Tablero	46:	0.548685
Tablero	47:	0.389254
Tablero	48:	0.450422
Tablero	49:	0.372859
Tablero	50:	0.002739
Tablero	51:	0.546248
Tablero	52:	0.762781
Tablero	53:	0.317973
Tablero	54:	0.966881
Tablero	55:	0.303248
Tablero	56:	1.524306
Tablero	57:	0.791530
Tablero	58:	0.848404
Tablero	59:	0.573049
Tablero	60:	0.707350
Tablero	61:	0.719517
Tablero	62:	1.268570
Tablero	63:	0.529656
Tablero	64:	0.258964
Tablero	65:	0.811786
Tablero	66:	0.695967
Tablero	67:	1.400598
Tablero	68:	0.893325
Tablero	69:	1.791068
Tablero	70:	0.858703
Tablero	71:	0.638703
Tablero	72:	0.498023
Tablero	73:	0.510801
Tablero	74:	0.385885
Tablero	75:	2.395607
Tablero	76:	0.908863
Tablero	77:	0.730093
Tablero	78:	0.631649
Tablero	79:	0.917014
Tablero	80:	0.638238
Tablero	81:	0.417851
Tablero	82:	0.192028
Tablero	83:	0.766867
Tablero	84:	0.484147
Tablero	85:	0.881626
Tablero	86:	0.424804
Tablero	87:	0.570809
Tablero	88:	0.793789
Tablero	89:	0.258951
Tablero	90:	0.828106
Tablero	91:	0.490015
Tablero	92:	0.833662
Tablero	93:	0.780450
Tablero	94:	0.624704
Tablero	95:	0.410303
Tablero	96:	0.608634
Tablero	97:	0.579104
Tablero	98:	0.788790
Tablero	99:	0.620276
Tablero	100:	0.090740
Tablero	101:	0.792564
Tablero	102:	0.330999
Tablero	103:	0.525822
Tablero	104:	0.811940

Tablero	105:	0.867326
Tablero	106:	2.723302
Tablero	107:	0.679031
Tablero	108:	0.491157
Tablero	109:	0.551043
Tablero	110:	0.611849
Tablero	111:	0.861395
Tablero	112:	0.904952
Tablero	113:	1.079928
Tablero	114:	0.567007
Tablero	115:	0.618219
Tablero	116:	0.520093
Tablero	117:	0.667485
Tablero	118:	0.900844
Tablero	119:	0.547929
Tablero	120:	1.167757
Tablero	121:	1.156967
Tablero	122:	0.608717
Tablero	123:	1.122540
Tablero	124:	0.690207
Tablero	125:	0.867443
Tablero	126:	1.010469
Tablero	127:	0.677774
Tablero	128:	0.737204
Tablero	129:	0.848333
Tablero	130:	0.068568
Tablero	131:	0.341502
Tablero	132:	0.702567
Tablero	133:	1.024008
Tablero	134:	1.562060
Tablero	135:	0.726296
Tablero	136:	0.318033
Tablero	137:	0.610995
Tablero	138:	0.468822
Tablero	139:	0.562057
Tablero	140:	0.495493
Tablero	141:	0.950428
Tablero	142:	0.620309
Tablero	143:	0.863209
Tablero	144:	0.320321
Tablero	145:	0.462580
Tablero	146:	1.157917
Tablero	147:	0.622424
Tablero	148:	1.046854
Tablero	149:	0.744397
Tablero	150:	0.414790
Tablero	151:	0.774519
Tablero	152:	1.188496
Tablero	153:	0.646987
Tablero	154:	0.618791
Tablero	155:	0.155733
Tablero	156:	0.768823
Tablero	157:	0.310509
Tablero	158:	0.664919
Tablero	159:	0.334271
Tablero	160:	0.626048
Tablero	161:	0.820983
Tablero	162:	0.855462
Tablero	163:	0.026404
Tablero	164:	0.710481
Tablero	165:	0.846321
Tablero	166:	0.082351

Tablero	167:	0.508180
Tablero	168:	0.247802
Tablero	169:	0.426137
Tablero	170:	0.734019
Tablero	171:	0.146969
Tablero	172:	0.151760
Tablero	173:	0.193376
Tablero	174:	0.789479
Tablero	175:	0.733520
Tablero	176:	0.209720
Tablero	177:	0.634296
Tablero	178:	0.859469
Tablero	179:	0.681936
Tablero	180:	0.219766
Tablero	181:	1.266530
Tablero	182:	0.439814
Tablero	183:	0.571104
Tablero	184:	0.616150
Tablero	185:	0.518734
Tablero	186:	0.755121
Tablero	187:	0.500536
Tablero	188:	0.837781
Tablero	189:	0.805170
Tablero	190:	0.600122
Tablero	191:	0.371672
Tablero	192:	3.080684
Tablero	193:	0.310304
Tablero	194:	0.629764
Tablero	195:	0.524782
Tablero	196:	0.904050
Tablero	197:	0.189332
Tablero	198:	0.676097
Tablero	199:	1.004518
Tablero	200:	0.814593
Tablero	201:	1.016040
Tablero	202:	0.367778
Tablero	203:	0.885250
Tablero	204:	0.526171
Tablero	205:	0.512804
Tablero	206:	0.200781
Tablero	207:	0.819742
Tablero	208:	0.484309
Tablero	209:	0.561106
Tablero	210:	0.376448
Tablero	211:	2.759974
Tablero	212:	0.173921
Tablero	213:	0.727791
Tablero	214:	0.579018
Tablero	215:	1.347481
Tablero	216:	0.449623
Tablero	217:	0.712951
Tablero	218:	0.567860
Tablero	219:	1.113242
Tablero	220:	0.147429
Tablero	221:	0.207800
Tablero	222:	0.493051
Tablero	223:	0.465745
Tablero	224:	1.041599
Tablero	225:	0.549470
Tablero	226:	1.442815
Tablero	227:	0.596686
Tablero	228:	3.199001

Tablero	229:	0.722851
Tablero	230:	0.859284
Tablero	231:	0.931899
Tablero	232:	0.377772
Tablero	233:	0.890048
Tablero	234:	0.068417
Tablero	235:	2.020071
Tablero	236:	1.020047
Tablero	237:	10.302543
Tablero	238:	0.856213
Tablero	239:	0.569386
Tablero	240:	1.111854
Tablero	241:	0.707039
Tablero	242:	0.385887
Tablero	243:	1.012334
Tablero	244:	0.734632
Tablero	245:	0.706838
Tablero	246:	0.963569
Tablero	247:	0.388722
Tablero	248:	0.697933
Tablero	249:	0.788527
Tablero	250:	0.831785
Tablero	251:	0.553451
Tablero	252:	0.233189
Tablero	253:	0.376392
Tablero	254:	0.583458
Tablero	255:	0.041568
Tablero	256:	1.102904
Tablero	257:	0.420136
Tablero	258:	0.465432
Tablero	259:	0.434255
Tablero	260:	0.381197
Tablero	261:	0.364638
Tablero	262:	0.766569
Tablero	263:	0.447456
Tablero	264:	2.750459
Tablero	265:	1.336200
Tablero	266:	1.283269
Tablero	267:	1.193094
Tablero	268:	0.800752
Tablero	269:	0.473348
Tablero	270:	0.500407
Tablero	271:	0.554521
Tablero	272:	0.529655
Tablero	273:	0.592764
Tablero	274:	0.867128
Tablero	275:	0.783438
Tablero	276:	0.000000
Tablero	277:	0.592495
Tablero	278:	0.392454
Tablero	279:	1.263109
Tablero	280:	0.596971
Tablero	281:	0.545302
Tablero	282:	1.275467
Tablero	283:	1.156047
Tablero	284:	0.845032
Tablero	285:	0.401259
Tablero	286:	1.045417
Tablero	287:	0.937567
Tablero	288:	1.310208
Tablero	289:	0.424033
Tablero	290:	2.036575

Tablero	291:	0.468830
Tablero	292:	1.266530
Tablero	293:	0.683238
Tablero	294:	0.353046
Tablero	295:	0.537465
Tablero	296:	0.502897
Tablero	297:	1.058571
Tablero	298:	0.411061
Tablero	299:	0.712944
Tablero	300:	0.325871
Tablero	301:	0.654560
Tablero	302:	1.355170
Tablero	303:	0.551830
Tablero	304:	0.906171
Tablero	305:	0.084980
Tablero	306:	0.700076
Tablero	307:	0.470792
Tablero	308:	1.288711
Tablero	309:	0.616557
Tablero	310:	0.508391
Tablero	311:	1.071148
Tablero	312:	0.334492
Tablero	313:	0.000000
Tablero	314:	0.404924
Tablero	315:	0.370322
Tablero	316:	0.680210
Tablero	317:	0.226711
Tablero	318:	0.631759
Tablero	319:	0.305929
Tablero	320:	1.172808
Tablero	321:	0.480642
Tablero	322:	0.173104
Tablero	323:	0.154335
Tablero	324:	1.386929
Tablero	325:	0.500336
Tablero	326:	0.860468
Tablero	327:	0.253213
Tablero	328:	0.996471
Tablero	329:	0.969468
Tablero	330:	0.571721
Tablero	331:	0.396573
Tablero	332:	1.126511
Tablero	333:	1.908468
Tablero	334:	0.209872
Tablero	335:	0.431206
Tablero	336:	0.321118
Tablero	337:	0.353946
Tablero	338:	0.387553
Tablero	339:	0.607856
Tablero	340:	0.411820
Tablero	341:	1.537420
Tablero	342:	0.725923
Tablero	343:	0.065261
Tablero	344:	0.671071
Tablero	345:	0.569051
Tablero	346:	0.000000
Tablero	347:	0.615340
Tablero	348:	0.712951
Tablero	349:	0.458508
Tablero	350:	0.677893
Tablero	351:	0.400893
Tablero	352:	0.104684

Tablero	353:	0.575781
Tablero	354:	0.553596
Tablero	355:	0.690509
Tablero	356:	0.253907
Tablero	357:	0.364059
Tablero	358:	0.722519
Tablero	359:	0.698826
Tablero	360:	0.525480
Tablero	361:	0.378039
Tablero	362:	1.231656
Tablero	363:	0.447646
Tablero	364:	0.544194
Tablero	365:	0.470104
Tablero	366:	0.486122
Tablero	367:	0.452400
Tablero	368:	0.753533
Tablero	369:	0.722205
Tablero	370:	0.943184
Tablero	371:	0.800147
Tablero	372:	0.600868
Tablero	373:	1.103289
Tablero	374:	0.548492
Tablero	375:	0.263564
Tablero	376:	0.009974
Tablero	377:	0.000000
Tablero	378:	0.956627
Tablero	379:	1.046804
Tablero	380:	0.582931
Tablero	381:	0.411547
Tablero	382:	2.174370
Tablero	383:	0.393784
Tablero	384:	0.497724
Tablero	385:	0.255008
Tablero	386:	0.252226
Tablero	387:	0.459145
Tablero	388:	0.183558
Tablero	389:	0.694606
Tablero	390:	0.069740
Tablero	391:	1.067098
Tablero	392:	0.557249
Tablero	393:	0.592086
Tablero	394:	0.404711
Tablero	395:	0.859951
Tablero	396:	0.342215
Tablero	397:	0.168242
Tablero	398:	0.474449
Tablero	399:	0.726627
Tablero	400:	0.428010
Tablero	401:	1.140315
Tablero	402:	1.162045
Tablero	403:	0.657663
Tablero	404:	0.368911
Tablero	405:	0.756762
Tablero	406:	0.534094
Tablero	407:	0.303795
Tablero	408:	4.257114
Tablero	409:	0.666596
Tablero	410:	3.080684
Tablero	411:	0.562144
Tablero	412:	0.460734
Tablero	413:	0.839352
Tablero	414:	0.432399

Tablero	415:	0.000000
Tablero	416:	0.603646
Tablero	417:	0.067288
Tablero	418:	0.567419
Tablero	419:	1.192266
Tablero	420:	0.733079
Tablero	421:	3.139042
Tablero	422:	1.069242
Tablero	423:	0.192067
Tablero	424:	0.738265
Tablero	425:	0.522100
Tablero	426:	0.197889
Tablero	427:	0.606228
Tablero	428:	0.703120
Tablero	429:	0.593153
Tablero	430:	0.590330
Tablero	431:	0.799902
Tablero	432:	0.690476
Tablero	433:	0.168313
Tablero	434:	0.484671
Tablero	435:	1.357832
Tablero	436:	0.133300
Tablero	437:	0.288255
Tablero	438:	0.565138
Tablero	439:	0.593545
Tablero	440:	4.257114
Tablero	441:	0.153310
Tablero	442:	0.465117
Tablero	443:	0.540266
Tablero	444:	0.409558
Tablero	445:	0.618394
Tablero	446:	0.635342
Tablero	447:	0.673025
Tablero	448:	1.035995
Tablero	449:	0.418138
Tablero	450:	0.744332
Tablero	451:	0.656531
Tablero	452:	1.145974
Tablero	453:	0.349119
Tablero	454:	1.059630
Tablero	455:	1.965147
Tablero	456:	0.531901
Tablero	457:	0.408989
Tablero	458:	0.566340
Tablero	459:	0.678974
Tablero	460:	0.541837
Tablero	461:	1.161477
Tablero	462:	0.309920
Tablero	463:	0.386481
Tablero	464:	1.607789
Tablero	465:	0.883342
Tablero	466:	3.611145
Tablero	467:	0.709792
Tablero	468:	0.451474
Tablero	469:	0.188698
Tablero	470:	0.338883
Tablero	471:	1.192250
Tablero	472:	0.238901
Tablero	473:	0.843948
Tablero	474:	0.837058
Tablero	475:	0.594751
Tablero	476:	0.614977

Tablero	477:	0.375003
Tablero	478:	0.890487
Tablero	479:	0.691473
Tablero	480:	0.514140
Tablero	481:	0.657713
Tablero	482:	0.000000
Tablero	483:	0.278030
Tablero	484:	0.898548
Tablero	485:	0.548945
Tablero	486:	0.485307
Tablero	487:	0.943738
Tablero	488:	0.851006
Tablero	489:	1.002591
Tablero	490:	0.983550
Tablero	491:	1.520343
Tablero	492:	0.270744
Tablero	493:	0.905346
Tablero	494:	0.983479
Tablero	495:	1.592916
Tablero	496:	0.380368
Tablero	497:	10.302543
Tablero	498:	0.840622
Tablero	499:	0.402449
Tablero	500:	0.051678
Tablero	501:	0.425826
Tablero	502:	1.704158
Tablero	503:	0.759835
Tablero	504:	1.457992
Tablero	505:	0.452980
Tablero	506:	0.134840
Tablero	507:	0.554990
Tablero	508:	0.545591
Tablero	509:	0.694536
Tablero	510:	0.805974
Tablero	511:	0.647136
Tablero	512:	0.416226
Tablero	513:	0.242904
Tablero	514:	0.281828
Tablero	515:	0.618407
Tablero	516:	0.376751
Tablero	517:	0.369725
Tablero	518:	0.580846
Tablero	519:	0.701411
Tablero	520:	0.429145
Tablero	521:	1.303530
Tablero	522:	0.300009
Tablero	523:	1.083128
Tablero	524:	0.760365
Tablero	525:	0.615613
Tablero	526:	0.563929
Tablero	527:	1.177853
Tablero	528:	0.602964
Tablero	529:	0.812227
Tablero	530:	0.916588
Tablero	531:	0.881286
Tablero	532:	0.338467
Tablero	533:	0.473901
Tablero	534:	0.768634
Tablero	535:	0.990027
Tablero	536:	0.298606
Tablero	537:	0.828293
Tablero	538:	0.488106

Tablero	539:	0.146430
Tablero	540:	0.511376
Tablero	541:	0.915639
Tablero	542:	0.402324
Tablero	543:	0.318056
Tablero	544:	0.093966
Tablero	545:	0.602096
Tablero	546:	0.598602
Tablero	547:	0.966698
Tablero	548:	0.224568
Tablero	549:	0.353909
Tablero	550:	0.716019
Tablero	551:	0.247440
Tablero	552:	0.601188
Tablero	553:	0.838830
Tablero	554:	0.529032
Tablero	555:	3.080684
Tablero	556:	1.319305
Tablero	557:	0.670131
Tablero	558:	0.924083
Tablero	559:	0.979894
Tablero	560:	0.850740
Tablero	561:	0.554439
Tablero	562:	1.953941
Tablero	563:	0.446922
Tablero	564:	1.995439
Tablero	565:	0.521941
Tablero	566:	1.373116
Tablero	567:	0.727018
Tablero	568:	0.431893
Tablero	569:	0.611243
Tablero	570:	3.639429
Tablero	571:	0.560044
Tablero	572:	0.000000
Tablero	573:	0.512570
Tablero	574:	1.080002
Tablero	575:	0.662296
Tablero	576:	0.511977
Tablero	577:	0.741933
Tablero	578:	0.719038
Tablero	579:	0.855942
Tablero	580:	1.118714
Tablero	581:	0.672956
Tablero	582:	0.594912
Tablero	583:	0.201712
Tablero	584:	0.364979
Tablero	585:	0.736505
Tablero	586:	0.947549
Tablero	587:	0.529666
Tablero	588:	0.486374
Tablero	589:	2.095681
Tablero	590:	0.167037
Tablero	591:	0.275966
Tablero	592:	0.268017
Tablero	593:	0.340555
Tablero	594:	0.660234
Tablero	595:	0.564360
Tablero	596:	0.544885
Tablero	597:	0.327274
Tablero	598:	0.520840
Tablero	599:	0.630910
Tablero	600:	0.425351

Tablero	601:	1.174204
Tablero	602:	0.686034
Tablero	603:	0.570728
Tablero	604:	0.302613
Tablero	605:	0.452523
Tablero	606:	0.839166
Tablero	607:	0.277611
Tablero	608:	0.738085
Tablero	609:	0.984675
Tablero	610:	0.753927
Tablero	611:	1.178534
Tablero	612:	0.000000
Tablero	613:	0.366209
Tablero	614:	0.273540
Tablero	615:	1.193499
Tablero	616:	0.666977
Tablero	617:	1.117075
Tablero	618:	0.642345
Tablero	619:	0.476505
Tablero	620:	0.208550
Tablero	621:	1.483451
Tablero	622:	0.270511
Tablero	623:	0.906654
Tablero	624:	0.564443
Tablero	625:	0.680054
Tablero	626:	0.891394
Tablero	627:	0.337164
Tablero	628:	0.148181
Tablero	629:	0.917436
Tablero	630:	0.935929
Tablero	631:	0.753196
Tablero	632:	0.723929
Tablero	633:	1.213013
Tablero	634:	1.374161
Tablero	635:	0.293329
Tablero	636:	0.811652
Tablero	637:	0.704532
Tablero	638:	0.609138
Tablero	639:	0.723269
Tablero	640:	0.887263
Tablero	641:	0.354745
Tablero	642:	0.387672
Tablero	643:	0.878552
Tablero	644:	0.364306
Tablero	645:	0.692750
Tablero	646:	0.866791
Tablero	647:	0.262286
Tablero	648:	0.397303
Tablero	649:	0.822904
Tablero	650:	0.609766
Tablero	651:	0.777665
Tablero	652:	0.296362
Tablero	653:	1.227523
Tablero	654:	0.367505
Tablero	655:	0.272440
Tablero	656:	0.341586
Tablero	657:	0.290127
Tablero	658:	1.487698
Tablero	659:	0.535014
Tablero	660:	0.899097
Tablero	661:	0.740522
Tablero	662:	0.685987

Tablero	663:	0.627364
Tablero	664:	0.535544
Tablero	665:	1.680137
Tablero	666:	0.000000
Tablero	667:	0.719039
Tablero	668:	1.050467
Tablero	669:	0.602397
Tablero	670:	4.528621
Tablero	671:	0.541276
Tablero	672:	0.605807
Tablero	673:	0.910736
Tablero	674:	0.495267
Tablero	675:	0.673464
Tablero	676:	0.594334
Tablero	677:	0.419678
Tablero	678:	0.707630
Tablero	679:	0.504516
Tablero	680:	0.226438
Tablero	681:	0.423342
Tablero	682:	0.353831
Tablero	683:	0.483453
Tablero	684:	0.756941
Tablero	685:	0.000000
Tablero	686:	0.822446
Tablero	687:	0.503151
Tablero	688:	0.553753
Tablero	689:	0.851041
Tablero	690:	0.830785
Tablero	691:	0.322015
Tablero	692:	0.597870
Tablero	693:	1.145459
Tablero	694:	0.512999
Tablero	695:	0.637979
Tablero	696:	0.057256
Tablero	697:	1.257796
Tablero	698:	0.384965
Tablero	699:	0.586477
Tablero	700:	0.321129
Tablero	701:	0.433209
Tablero	702:	0.349987
Tablero	703:	0.920121
Tablero	704:	0.755121
Tablero	705:	0.490723
Tablero	706:	0.927444
Tablero	707:	0.546893
Tablero	708:	0.426057
Tablero	709:	0.510152
Tablero	710:	0.530328
Tablero	711:	0.527282
Tablero	712:	0.217127
Tablero	713:	0.974623
Tablero	714:	0.529492
Tablero	715:	0.930346
Tablero	716:	1.098235
Tablero	717:	0.931072
Tablero	718:	1.043986
Tablero	719:	0.104917
Tablero	720:	1.055217
Tablero	721:	0.557688
Tablero	722:	0.860710
Tablero	723:	0.364349
Tablero	724:	1.243069

Tablero	725:	2.814810
Tablero	726:	0.208509
Tablero	727:	0.433370
Tablero	728:	0.000000
Tablero	729:	1.012861
Tablero	730:	0.245177
Tablero	731:	0.620225
Tablero	732:	0.151556
Tablero	733:	1.881446
Tablero	734:	0.484849
Tablero	735:	0.475837
Tablero	736:	0.732963
Tablero	737:	0.480843
Tablero	738:	0.737985
Tablero	739:	0.385771
Tablero	740:	0.580586
Tablero	741:	0.491964
Tablero	742:	0.520032
Tablero	743:	1.188007
Tablero	744:	0.558501
Tablero	745:	0.694294
Tablero	746:	0.682528
Tablero	747:	0.909253
Tablero	748:	1.031914
Tablero	749:	0.536174
Tablero	750:	0.627110
Tablero	751:	0.364466
Tablero	752:	0.182249
Tablero	753:	0.684368
Tablero	754:	0.689726
Tablero	755:	0.641345
Tablero	756:	1.007304
Tablero	757:	0.050424
Tablero	758:	1.833431
Tablero	759:	0.454295
Tablero	760:	0.694148
Tablero	761:	0.286161
Tablero	762:	0.777660
Tablero	763:	0.198563
Tablero	764:	0.153063
Tablero	765:	0.322088
Tablero	766:	0.552253
Tablero	767:	1.099148
Tablero	768:	0.430477
Tablero	769:	0.547596
Tablero	770:	10.302543
Tablero	771:	0.923697
Tablero	772:	1.902370
Tablero	773:	0.487837
Tablero	774:	0.403899
Tablero	775:	1.154725
Tablero	776:	3.199001
Tablero	777:	0.254135
Tablero	778:	0.876148
Tablero	779:	2.515319
Tablero	780:	0.190246
Tablero	781:	1.882035
Tablero	782:	0.832337
Tablero	783:	0.722585
Tablero	784:	0.480901
Tablero	785:	0.379343
Tablero	786:	10.302543

Tablero	787:	0.449063
Tablero	788:	0.906621
Tablero	789:	1.018734
Tablero	790:	0.628240
Tablero	791:	0.861018
Tablero	792:	1.040724
Tablero	793:	0.727484
Tablero	794:	0.124483
Tablero	795:	0.000000
Tablero	796:	0.615926
Tablero	797:	0.803280
Tablero	798:	0.053357
Tablero	799:	3.080684
Tablero	800:	1.223869
Tablero	801:	0.767610
Tablero	802:	0.411782
Tablero	803:	0.523315
Tablero	804:	1.272688
Tablero	805:	0.344926
Tablero	806:	0.727741
Tablero	807:	0.706689
Tablero	808:	0.156326
Tablero	809:	1.701826
Tablero	810:	0.457954
Tablero	811:	0.791132
Tablero	812:	0.682294
Tablero	813:	0.884276
Tablero	814:	0.750050
Tablero	815:	0.677578
Tablero	816:	0.087456
Tablero	817:	0.507271
Tablero	818:	0.712018
Tablero	819:	0.490381
Tablero	820:	0.610590
Tablero	821:	0.521762
Tablero	822:	1.183395
Tablero	823:	0.711039
Tablero	824:	0.678643
Tablero	825:	0.805965
Tablero	826:	0.799145
Tablero	827:	0.638263
Tablero	828:	0.361681
Tablero	829:	0.831669
Tablero	830:	1.129149
Tablero	831:	0.032552
Tablero	832:	1.001479
Tablero	833:	0.959035
Tablero	834:	0.669712
Tablero	835:	0.515224
Tablero	836:	0.621669
Tablero	837:	0.766400
Tablero	838:	0.414691
Tablero	839:	1.168417
Tablero	840:	1.261131
Tablero	841:	0.216319
Tablero	842:	1.282053
Tablero	843:	0.374468
Tablero	844:	0.859652
Tablero	845:	0.705987
Tablero	846:	3.139042
Tablero	847:	0.723420
Tablero	848:	0.816592

Tablero	849:	0.831618
Tablero	850:	0.290043
Tablero	851:	0.317950
Tablero	852:	0.626316
Tablero	853:	1.281389
Tablero	854:	0.802304
Tablero	855:	0.493577
Tablero	856:	0.299688
Tablero	857:	0.402996
Tablero	858:	0.597324
Tablero	859:	0.603680
Tablero	860:	0.383643
Tablero	861:	0.603713
Tablero	862:	0.842977
Tablero	863:	0.650293
Tablero	864:	0.700818
Tablero	865:	0.820983
Tablero	866:	0.651892
Tablero	867:	0.856280
Tablero	868:	1.123394
Tablero	869:	0.860745
Tablero	870:	0.238981
Tablero	871:	0.581039
Tablero	872:	10.302543
Tablero	873:	0.609129
Tablero	874:	0.806820
Tablero	875:	0.702656
Tablero	876:	0.419700
Tablero	877:	0.265190
Tablero	878:	0.000000
Tablero	879:	0.503458
Tablero	880:	0.662649
Tablero	881:	0.448124
Tablero	882:	0.598765
Tablero	883:	0.729799
Tablero	884:	1.605201
Tablero	885:	0.914451
Tablero	886:	0.705062
Tablero	887:	0.419933
Tablero	888:	0.428642
Tablero	889:	0.794373
Tablero	890:	0.296213
Tablero	891:	0.694809
Tablero	892:	0.732452
Tablero	893:	1.014146
Tablero	894:	0.594802
Tablero	895:	0.654481
Tablero	896:	0.586236
Tablero	897:	0.827007
Tablero	898:	0.712147
Tablero	899:	1.576962
Tablero	900:	0.662849
Tablero	901:	0.256605
Tablero	902:	0.176005
Tablero	903:	1.002614
Tablero	904:	0.212438
Tablero	905:	0.390051
Tablero	906:	0.683089
Tablero	907:	0.284368
Tablero	908:	0.357021
Tablero	909:	0.276129
Tablero	910:	0.368085

Tablero	911:	0.654578
Tablero	912:	0.806939
Tablero	913:	1.100803
Tablero	914:	1.261959
Tablero	915:	0.663494
Tablero	916:	0.339171
Tablero	917:	0.502972
Tablero	918:	0.496041
Tablero	919:	0.489858
Tablero	920:	0.534503
Tablero	921:	0.693140
Tablero	922:	0.645056
Tablero	923:	0.799336
Tablero	924:	0.331808
Tablero	925:	0.510589
Tablero	926:	1.188385
Tablero	927:	0.000000
Tablero	928:	0.000000
Tablero	929:	0.641361
Tablero	930:	0.547624
Tablero	931:	1.670926
Tablero	932:	1.215964
Tablero	933:	0.907223
Tablero	934:	0.876569
Tablero	935:	0.100838
Tablero	936:	1.206640
Tablero	937:	0.623448
Tablero	938:	0.000000
Tablero	939:	0.821565
Tablero	940:	0.539915
Tablero	941:	0.818271
Tablero	942:	0.766535
Tablero	943:	1.195559
Tablero	944:	0.688682
Tablero	945:	0.534964
Tablero	946:	1.881380
Tablero	947:	0.373271
Tablero	948:	0.374557
Tablero	949:	0.487276
Tablero	950:	0.587930
Tablero	951:	0.568777
Tablero	952:	1.012035
Tablero	953:	1.351071
Tablero	954:	0.598749
Tablero	955:	0.382553
Tablero	956:	0.489055
Tablero	957:	0.760109
Tablero	958:	0.808994
Tablero	959:	0.277499
Tablero	960:	0.937989
Tablero	961:	0.709260
Tablero	962:	0.640396
Tablero	963:	0.928153
Tablero	964:	0.381722
Tablero	965:	0.407021
Tablero	966:	1.084421
Tablero	967:	1.399273
Tablero	968:	0.876404
Tablero	969:	0.375530
Tablero	970:	0.328526
Tablero	971:	0.875369
Tablero	972:	0.272029

```

Tablero 973: 5.745603
Tablero 974: 0.351794
Tablero 975: 2.527196
Tablero 976: 0.932951
Tablero 977: 2.240921
Tablero 978: 0.097595
Tablero 979: 0.410684
Tablero 980: 0.630676
Tablero 981: 0.618082
Tablero 982: 0.884246
Tablero 983: 0.084195
Tablero 984: 0.449668
Tablero 985: 1.076867
Tablero 986: 4.257114
Tablero 987: 0.111889
Tablero 988: 0.639725
Tablero 989: 0.881919
Tablero 990: 0.534090
Tablero 991: 0.326607
Tablero 992: 0.000000
Tablero 993: 0.606295
Tablero 994: 0.538130
Tablero 995: 0.304828
Tablero 996: 0.197332
Tablero 997: 0.414984
Tablero 998: 0.993786
Tablero 999: 0.406848
Tablero 1000: 0.194889

```

```

def plot_bsp_histogram(BSP_HISTOGRAMA, sae_features, bsp_pieces_names, bsp_pieces_gt_tensor,
                       best_features, best_thresholds, best_fls, n_bins=20):

    # Extraer datos de la BSP
    bsp_idx_hist = bsp_pieces_names.index(BSP_HISTOGRAMA)
    neurona_hist = best_features[bsp_idx_hist]
    threshold_hist = best_thresholds[bsp_idx_hist]
    fl_hist = best_fls[bsp_idx_hist]
    f_max_hist = sae_features[:, neurona_hist].max().item()

    # Activaciones y ground truth
    activaciones = sae_features[:, neurona_hist].cpu().numpy()
    gt = bsp_pieces_gt_tensor[:, bsp_idx_hist].cpu().numpy()
    umbral = threshold_hist * f_max_hist

    # Clasificar
    predicho = (activaciones > umbral).astype(int)
    tp_mask = (gt == 1) & (predicho == 1)
    fp_mask = (gt == 0) & (predicho == 1)
    tn_mask = (gt == 0) & (predicho == 0)
    fn_mask = (gt == 1) & (predicho == 0)

    # Bins
    bins = np.linspace(0, activaciones.max(), n_bins + 1)
    bin_centers = (bins[:-1] + bins[1:]) / 2
    bin_width = bins[1] - bins[0]

    tp_counts, _ = np.histogram(activaciones[tp_mask], bins=bins)
    fp_counts, _ = np.histogram(activaciones[fp_mask], bins=bins)
    tn_counts, _ = np.histogram(activaciones[tn_mask], bins=bins)
    fn_counts, _ = np.histogram(activaciones[fn_mask], bins=bins)

    left_mask = bin_centers <= umbral

```

```

right_mask = bin_centers > umbral

# Graficar
fig, ax = plt.subplots(figsize=(14, 6))

ax.bar(bin_centers[left_mask], tn_counts[left_mask],
       width=bin_width, color='steelblue', alpha=0.9,
       label='TN (GT=0, pred=0)', align='center')
ax.bar(bin_centers[left_mask], fn_counts[left_mask],
       width=bin_width, bottom=tn_counts[left_mask],
       color='lightcoral', alpha=0.9,
       label='FN (GT=1, pred=0)', align='center')
ax.bar(bin_centers[right_mask], tp_counts[right_mask],
       width=bin_width, color='green', alpha=0.9,
       label='TP (GT=1, pred=1)', align='center')
ax.bar(bin_centers[right_mask], fp_counts[right_mask],
       width=bin_width, bottom=tp_counts[right_mask],
       color='orange', alpha=0.9,
       label='FP (GT=0, pred=1)', align='center')

# Números TN
for i, x in enumerate(bin_centers[left_mask]):
    total = tn_counts[left_mask][i] + fn_counts[left_mask][i]
    if tn_counts[left_mask][i] > 0:
        ax.text(x, total + 0.3, str(tn_counts[left_mask][i]),
                ha='center', va='bottom', fontsize=7, color='steelblue')

# Números TP
for i, x in enumerate(bin_centers[right_mask]):
    total = tp_counts[right_mask][i] + fp_counts[right_mask][i]
    if tp_counts[right_mask][i] > 0:
        ax.text(x, total + 0.3, str(tp_counts[right_mask][i]),
                ha='center', va='bottom', fontsize=7, color='green')

# Umbral
ax.axvline(x=umbral, color='red', linewidth=2, linestyle='--',
          label=f'Umbral = {threshold_hist:.1f} × {f_max_hist:.3f} = {umbral:.4f}')

ax.set_xlabel('Activación de la neurona')
ax.set_ylabel('Conteo de tableros')
ax.set_title(f'Histograma BSP: {BSP_HISTOGRAMA} | Neurona: {neurona_hist} | F1: {f1_hist:.4f}')
ax.legend()
ax.text(umbral * 0.5, ax.get_ylim()[1] * 0.95, 'predicho = 0',
       ha='center', fontsize=10, color='gray')
ax.text(umbral + (activaciones.max() - umbral) * 0.5, ax.get_ylim()[1] * 0.95, 'predicho = 1',
       ha='center', fontsize=10, color='gray')

plt.tight_layout()
plt.show()

print(f"\nResumen:")
print(f"  TP: {tp_mask.sum()} | FP: {fp_mask.sum()}")
print(f"  TN: {tn_mask.sum()} | FN: {fn_mask.sum()}")
print(f"  F1: {f1_hist:.4f}")

print(" Función plot_bsp_histogram definida")

```

Función plot_bsp_histogram definida

```

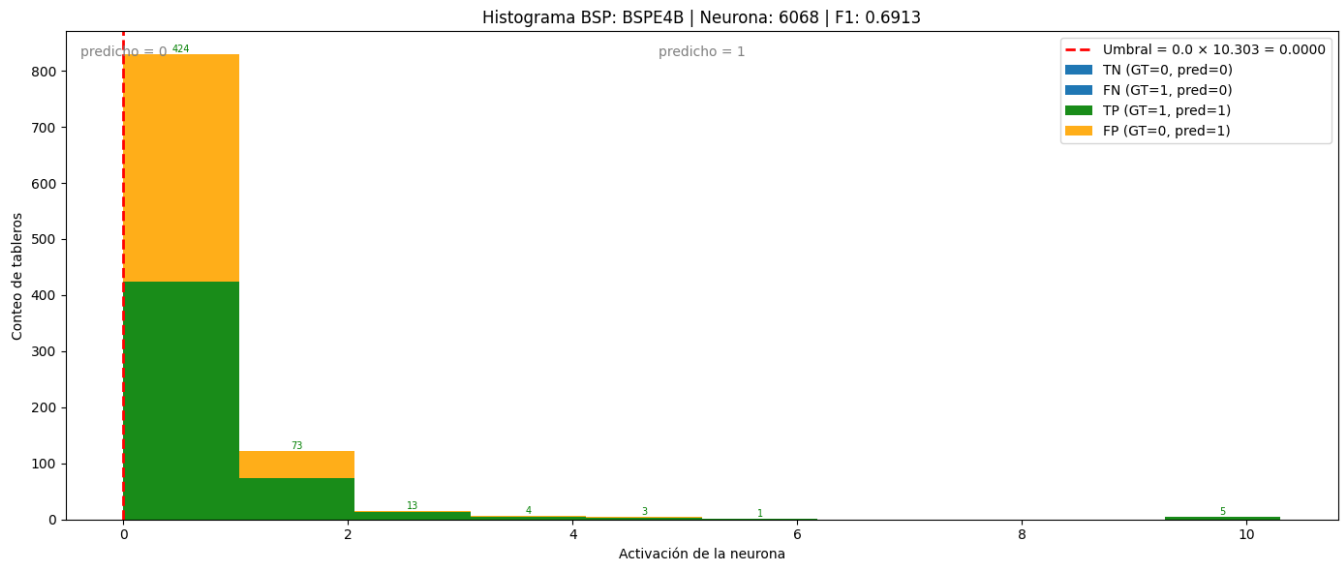
# >>> CAMBIA AQUÍ LA BSP QUE QUIERES VISUALIZAR <
import matplotlib.pyplot as plt

```

```

plot_bsp_histogram(
    BSP_HISTOGRAMA      = "BSPE4B",
    sae_features         = sae_features,
    bsp_pieces_names    = bsp_pieces_names,
    bsp_pieces_gt_tensor = bsp_pieces_gt_tensor,
    best_features        = best_features,
    best_thresholds      = best_thresholds,
    best_f1s             = best_f1s,
    n_bins               = 10
)

```



Resumen:

TP: 523 | FP: 459
 TN: 10 | FN: 8
 F1: 0.6913

```

import matplotlib.pyplot as plt
import numpy as np

def plot_f1_distribution(best_f1s, bsp_pieces_names, n_bins=10):
    """
    Gráfico 1: Distribución de F1 scores por BSP.

    Args:
        best_f1s: array numpy con el mejor F1 score de cada BSP
        bsp_pieces_names: lista con los nombres de las BSPs
        n_bins: número de bins del histograma
    """
    fig, ax = plt.subplots(figsize=(10, 6))

    bins = np.linspace(0, 1, n_bins + 1)
    counts, edges = np.histogram(best_f1s, bins=bins)
    bin_centers = (edges[:-1] + edges[1:]) / 2
    bin_width = edges[1] - edges[0]

    bars = ax.bar(
        bin_centers, counts,
        width=bin_width * 0.85,
        color='steelblue',
        edgecolor='white',

```

```

        linewidth=0.8
    )

    # Etiquetas encima de cada barra
    for bar, count in zip(bars, counts):
        if count > 0:
            ax.text(
                bar.get_x() + bar.get_width() / 2,
                bar.get_height() + 0.3,
                str(count),
                ha='center', va='bottom', fontsize=9
            )

    ax.set_xlabel('F1 Score', fontsize=12)
    ax.set_ylabel('Cantidad de BSPs', fontsize=12)
    ax.set_title('Distribución de F1 Scores por BSP', fontsize=14)
    ax.set_xticks(bins)
    ax.set_xticklabels([f'{b:.1f}' for b in bins], rotation=45)
    ax.set_xlim(0, 1)

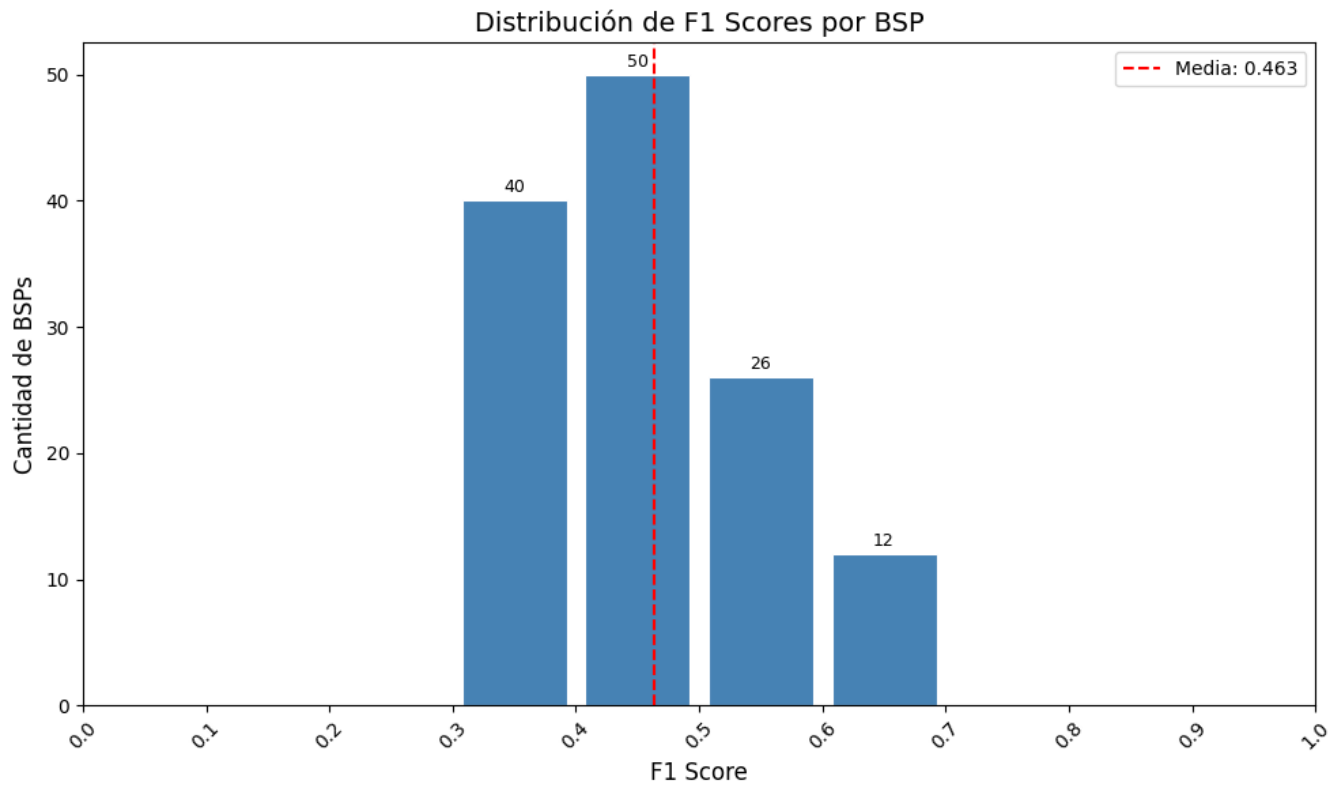
    # Línea de la media
    mean_f1 = best_f1s.mean()
    ax.axvline(mean_f1, color='red', linestyle='--', linewidth=1.5, label=f'Media: {mean_f1:.3f}')
    ax.legend(fontsize=10)

    plt.tight_layout()
    plt.show()

    print(f"Total BSPs: {len(best_f1s)}")
    print(f"Media F1: {mean_f1:.4f}")
    print(f"BSPs con F1 = 0: {(best_f1s == 0).sum()}")
    print(f"BSPs con F1 > 0.5: {(best_f1s > 0.5).sum()}")
    print(f"BSPs con F1 > 0.9: {(best_f1s > 0.9).sum()}")

# Ejecutar
plot_f1_distribution(best_f1s, bsp_pieces_names)

```



Total BSPs: 128
 Media F1: 0.4630
 BSPs con F1 = 0: 0
 BSPs con F1 > 0.5: 38
 BSPs con F1 > 0.9: 0

```
# Extraer f_max de las features activas
f_max = sae_features.max(dim=0)[0] # (n_features,)
```

```
def plot_max_activation_distribution(best_features, f_max, bsp_pieces_names, n_bins=10):
    """
    Gráfico 2: Distribución de activación máxima de la feature ganadora por BSP.

    Args:
        best_features: array numpy con el índice de la feature ganadora de cada BSP
        f_max: tensor con la activación máxima de cada feature
        bsp_pieces_names: lista con los nombres de las BSPs
        n_bins: número de bins del histograma
    """
    # Obtener f_max de la feature ganadora de cada BSP
    # Ignorar BSPs sin feature ganadora (best_features == -1)
    valid_mask = best_features >= 0
    valid_features = best_features[valid_mask]
    f_max_winners = f_max[valid_features].cpu().numpy()

    fig, ax = plt.subplots(figsize=(10, 6))

    bins = np.linspace(f_max_winners.min(), f_max_winners.max(), n_bins + 1)
    counts, edges = np.histogram(f_max_winners, bins=bins)
    bin_centers = (edges[:-1] + edges[1:]) / 2
    bin_width = edges[1] - edges[0]

    bars = ax.bar(
```

```

    bin_centers, counts,
    width=bin_width * 0.85,
    color='steelblue',
    edgecolor='white',
    linewidth=0.8
)

# Etiquetas encima de cada barra
for bar, count in zip(bars, counts):
    if count > 0:
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 0.3,
            str(count),
            ha='center', va='bottom', fontsize=9
        )

# Línea de la media
mean_fmax = f_max_winners.mean()
ax.axvline(mean_fmax, color='red', linestyle='--', linewidth=1.5,
            label=f'Media: {mean_fmax:.3f}')
ax.legend(fontsize=10)

ax.set_xlabel('Activación Máxima (f_max)', fontsize=12)
ax.set_ylabel('Cantidad de BSPs', fontsize=12)
ax.set_title('Distribución de Activación Máxima de Features Ganadoras', fontsize=14)
ax.set_xticks(edges)
ax.set_xticklabels([f'{b:.2f}' for b in edges], rotation=45)

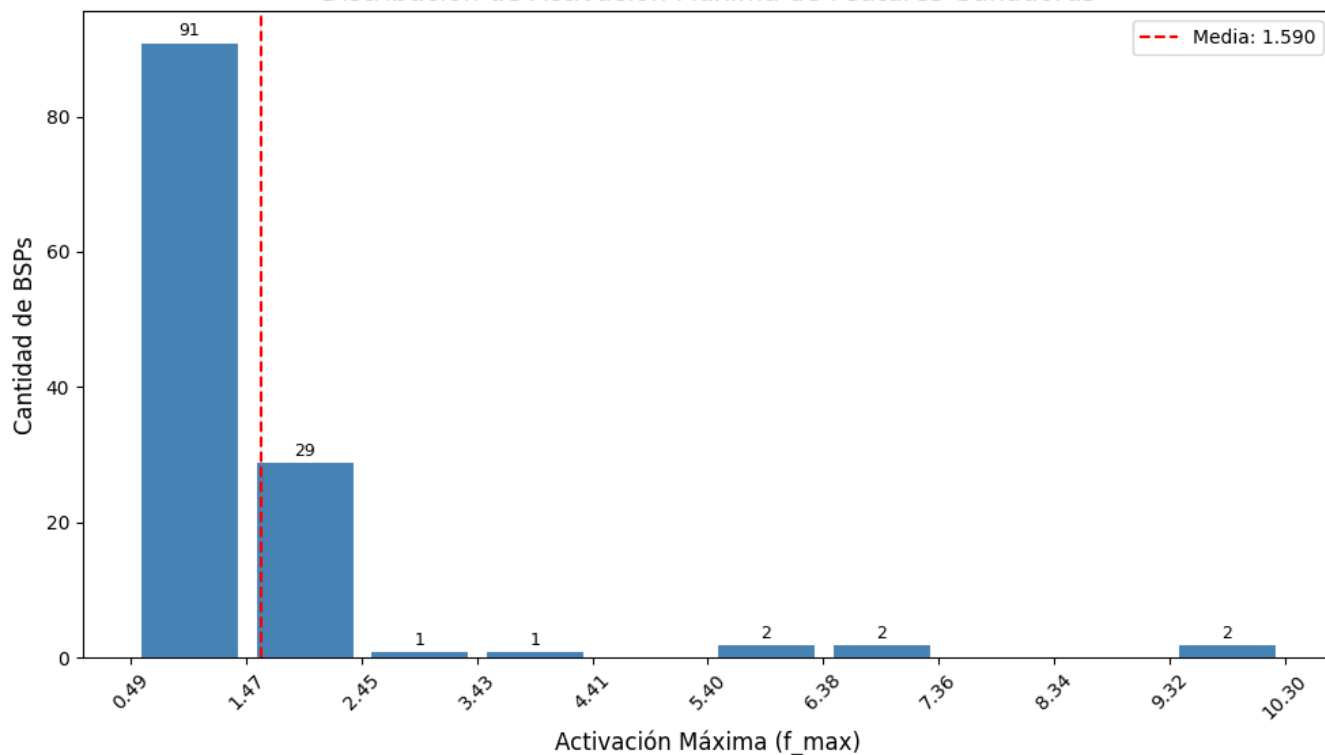
plt.tight_layout()
plt.show()

print(f"Total BSPs con feature ganadora: {valid_mask.sum()}")
print(f"BSPs sin feature ganadora: {(~valid_mask).sum()}")
print(f"Media f_max ganadora: {mean_fmax:.4f}")
print(f"Mínimo f_max: {f_max_winners.min():.4f}")
print(f"Máximo f_max: {f_max_winners.max():.4f}")

# Ejecutar
plot_max_activation_distribution(best_features, f_max, bsp_pieces_names)

```


Distribución de Activación Máxima de Features Ganadoras



Total BSPs con feature ganadora: 128

BSPs sin feature ganadora: 0

Media f_max ganadora: 1.5896

Mínimo f_max: 0.4878

Máximo f_max: 10.3025

```
def plot_threshold_distribution(best_thresholds, bsp_pieces_names):
    """
    Gráfico 3: Distribución de umbrales óptimos por BSP.

    Args:
        best_thresholds: array numpy con el umbral óptimo de cada BSP
        bsp_pieces_names: lista con los nombres de las BSPs
    """
    # Ignorar BSPs sin feature ganadora (best_thresholds == -1)
    valid_mask = best_thresholds >= 0
    valid_thresholds = best_thresholds[valid_mask]

    fig, ax = plt.subplots(figsize=(10, 6))

    # Los umbrales son discretos: 0.0, 0.1, 0.2, ..., 0.9
    threshold_values = np.arange(0.0, 1.0, 0.1)
    counts = [(np.abs(valid_thresholds - t) < 0.05).sum() for t in threshold_values]

    bars = ax.bar(
        threshold_values, counts,
        width=0.07,
        color='steelblue',
        edgecolor='white',
        linewidth=0.8
    )

    # Etiquetas encima de cada barra
```

```

for bar, count in zip(bars, counts):
    if count > 0:
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 0.3,
            str(count),
            ha='center', va='bottom', fontsize=9
        )

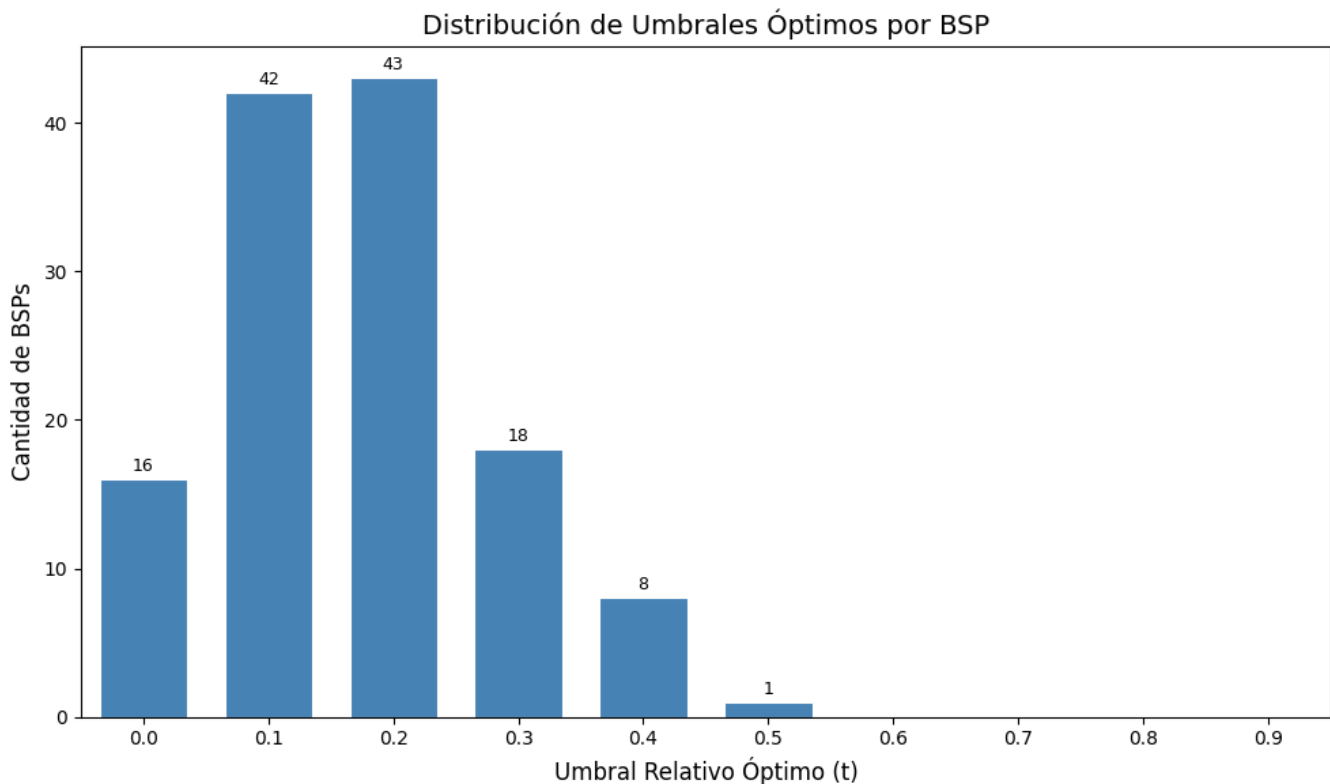
ax.set_xlabel('Umbral Relativo Óptimo (t)', fontsize=12)
ax.set_ylabel('Cantidad de BSPs', fontsize=12)
ax.set_title('Distribución de Umbrales Óptimos por BSP', fontsize=14)
ax.set_xticks(threshold_values)
ax.set_xticklabels([f'{t:.1f}' for t in threshold_values])
ax.set_xlim(-0.05, 0.95)

plt.tight_layout()
plt.show()

print(f"Total BSPs con umbral óptimo: {valid_mask.sum()}")
print(f"BSPs sin feature ganadora: {(~valid_mask).sum()}")
print(f"Umbral más frecuente: {threshold_values[np.argmax(counts)]:.1f}")

# Ejecutar
plot_threshold_distribution(best_thresholds, bsp_pieces_names)

```



Total BSPs con umbral óptimo: 128
 BSPs sin feature ganadora: 0
 Umbral más frecuente: 0.2

0.8 8. Guardar Resultados

```
# Guardar resultados en el directorio de métricas del experimento
results = {
    'coverage': coverage,
    'best_fis': best_fis,
    'best_features': best_features,
    'best_thresholds': best_thresholds,
    'bsp_names': bsp_pieces_names,
    'execution_time_seconds': elapsed_time
}

metrics_dir = get_metrics_dir(
    NAMING_META['experiment_base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    extras_id=extras_id
)
output_path = metrics_dir / f"coverage_{filter_suffix}.npz"
np.savez(output_path, **results)

print(f" Resultados guardados en:")
print(f"   {output_path}")
```

Resultados guardados en:

c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\sae_standard\sae_l1-decay_standard\500games\ex

0.9 9. Generar PDF con Quarto

```
import subprocess

# Guardar PDF en el mismo directorio que el .npz (metrics/)
pdf_dir = get_metrics_dir(
    NAMING_META['experiment_base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    extras_id=extras_id
)

pdf_name = get_report_name(
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    f'coverage_{filter_suffix}',
    extras_id=extras_id
)

notebook_name = 'coverage_gpu_optimized.ipynb'
output_path = pdf_dir / pdf_name

print(f' Generando PDF con Quarto...')
print(f' Archivo de salida: {output_path}')

# Quarto no acepta rutas en --output, solo nombre de archivo
```

```
# Se usa --output-dir para el directorio y --output solo para el nombre
result = subprocess.run(
    f'quarto render {notebook_name} --to pdf --output-dir "{pdf_dir}" --output {pdf_name}',
    shell=True,
    capture_output=True,
    text=True
)

if result.returncode == 0:
    print(f' PDF generado exitosamente: {output_path}')
else:
    print(f' Error al generar PDF:')
    print(result.stderr)
```

Generando PDF con Quarto...

Archivo de salida: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\sae_standard\sae_l1-decay_standard\500games\ex5\metrics\sae_standard_l1-decay_l6_500g_ex5_coverage_absolute.pdf
 PDF generado exitosamente: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\sae_standard\sae_l1-decay_standard\500games\ex5\metrics\sae_standard_l1-decay_l6_500g_ex5_coverage_absolute.pdf